

Universidad de Concepción

Inteligencia Artificial 2026-2: Tarea 1

Simón Pablo Serrano Luarte
Matrícula: 2024455419

September 28, 2026

Algoritmos usados

A continuación se presentan los algoritmos de búsqueda informada, no informada y genético, junto a algunos detalles de su implementación.

Detalles comunes

- los estados son de la forma (posición fila, posición columna, turno en el que llega a esa posición). Fue importante añadir la dimensión temporal para poder discriminar la calidad de la ruta si intentamos ir a una misma posicion en turnos distintos. Por ejemplo, si tenemos 2 agentes que desean ir a la misma casilla en el mismo turno, el 2do agente en llegar le tomaría varios turnos por la penalización de congestión, su mejor opción sería esperar un turno y luego moverse cuando no haya nadie, es decir, llega a la misma posición pero en un tiempo distinto. Esto es útil para UCS y A* que se guían por el costo.

- Se utilizan 2 pisos, donde el fuego puede aparecer en cualquiera de los dos y puede subir o bajar por la salida respectiva del piso. Sin embargo, se cuenta como sobreviviente si llega a la salida del piso (no tiene que bajar la "escalera"). La ejecución de los algoritmos por piso es secuencial y tomamos el máximo de los turnos del ultimo sobreviente de cada piso para definir el último sobreviviente del edificio entero.

- Cada agente tiene una posición global (tupla piso, y,x), pero los algoritmos trabajan con la versión 2D (y,x)

- Dado que los algoritmos buscan una salida "offline", solo volvemos a correrlos cuando el fuego se propaga, ya que el mapa cambia.

- Para manejar la congestión, usamos un diccionario de "reservas de posición", donde dado el proximo turno y la posición a la que se consulta, nos dice cuántos agentes se quieren mover ahí.

- Para penalizar el irse a una casilla con más agentes, en la lista de ruta del agente se inyecta la misma posición en función del costo por dicha congestión. Se usó como función de costo: 3(ocupantes) + 1.

- Se implementó un límite de profunidad en las posibles rutas, esto fue originalmente para evitar los loops infinitos de Greedy (por ejemplo, siempre decide esperar o de lo contrario toma valores de h más caros). Para consistencia con la comparación, se dejó en todos los algoritmos. Una consecuencia es cortar rutas que llevaban al agente a sobrevivir de forma prematura.

- Las estructuras de datos que guardan los estados (Queue y Heaps), además de guardar los costos (excepto la Queue de BFS) guardan la ruta de donde vino ese estado, ya que así podemos simular el movimiento.

Búsqueda no informada

BFS

Implementación clásica usando un deque como queue. Para ahorrar RAM, se marca como visitados los estados antes de meterlos a la cola, así evitamos que dada la expansión de un estado, se meta a la cola un estado que ya está en la cola. La solución encontrada por BFS cumple con ser la más corta respecto al número de casillas tomadas para llegar a la meta, mas no es óptima puesto que puede tomarle muchos turnos si existe congestión.

UCS

Implementación clásica usando un MINHEAP, sacando el estado de menor costo.

Búsqueda informada

La heurística utilizada fue la distancia Manhattan, la cual es consistente.

Búsqueda Greedy

Implementación clásica sujeta a las variaciones descritas en el apartado de Detalles comunes.

A*

Implementación clásica sujeta a las variaciones descritas en el apartado de Detalles comunes.

Genético

Implementado con ayuda de IA generativa (Gemini Pro).

- Individuo: se define como el orden en que cada agente ejecuta A*. Por ejemplo, si tenemos 2 agentes, podemos tener al individuo (0,1) y (1,0), donde el primer individuo es que el agente 0 ejecuta primero A*. Las implicancias son que el orden de ejecución podría afectar a la cantidad de turnos tomadas por el último sobreviviente. Un caso puede ser el siguiente: Tenemos dos agentes en un pasillo uno detrás del otro, si el que está detrás le toca primero, va a esperar 1 turno extra a diferencia de que si el que está primero avanza (el segundo agente ve vacía la siguiente casilla).
- Función de fitness: Para evaluar la calidad de un individuo, corremos A* respecto al orden que indica el ADN del individuo. Si hay sobrevivientes, definimos el costo como 1000(muertes) + turno del ultimo sobreviviente, así castigamos fuertemente al individuo si mueren los agentes y al sumar el turno del ultimo podemos discriminar entre individuos donde los agentes escaparon más rápido (que es mejor). Si no sobreviven todos los agentes,

el costo es 1000(muertes) - turno ultimo sobreviviente, de esta forma entre individuos donde todos mueren, es mejor uno donde más tiempo (turnos) duran vivos.

- Cruces: En cada generación se elijen los 2 de menor costo y pasan directamente a la siguiente, para completar los N - 2 individuos que nos quedan (con N el número de inviduos de la población), debemos hacer (N-2)/2 cruces, ya que cada cruce nos da 2 hijos. Para elegir los padres del cruce, formamos 2 torneos, donde cada torneo consta de elegir una muestra de 3 individuos de los N y elegir el mejor (menor costo). Para cruzar los padres elegimos un trozo del ADN (una subsecuencia de agentes) al azar, el padre 1 le hereda dicha secuencia al hijo 1 y el padre 2 al hijo 2, los genes (agentes) que faltaron en cada hijo, se rellenan según el orden en que aparacen en el otro padre.

- Mutación: Para cada hijo, con probabilidad de 0.05, cambiamos dos genes de posición al azar, es decir, dos agentes se cambian de orden en que ejecutan A* (ahora el otro lo ejecuta primero.)

Experimentación y resultados

Se realizaron dos experimentos, el primero fue para todos los algoritmos en cada mapa con las siguientes condiciones: 2 pisos, el fuego se propaga cada 3 turnos, límite de profundidad para los algortimos de 20 turnos y 4 agentes. La cantidad baja de agentes fue por el tiempo que toma el algoritmo genético, es por eso que se decidió tomar un segundo experimento con exactamente las mismas condiciones salvo con 80 agentes, pero solo para comparar los otros algoritmos. Se usó un total de 80 iteraciones para cada algoritmo y mapa.

Resultados experimento 1

- BFS:

Mapa	Tasa de supervivencia	Media turno	sd Turno	min Turno	max Turno
Mapa 1	42.19%	87.74	58.98	13	206
Mapa 2	75.31%	23.06	16.84	8	110
Mapa 3	77.81%	21.30	14.40	8	113

Table 1: Resultados BFS 1.

- UCS:

Mapa	Tasa de supervivencia	Media turno	sd Turno	min Turno	max Turno
Mapa 1	28.44%	114.46	58.78	18	233
Mapa 2	74.69%	26.59	21.67	14	113
Mapa 3	79.38%	20.75	13.75	10	107

Table 2: Resultados UCS 1.

- Búsqueda Greedy:

Mapa	Tasa de supervivencia	Media turno	sd Turno	min Turno	max Turno
Mapa 1	30.00%	70.90	66.25	8	230
Mapa 2	75.94%	22.76	14.20	9	92
Mapa 3	75.95%	19.38	9.57	8	77

Table 3: Resultados Búsqueda Greedy 1.

- A*:

Mapa	Tasa de supervivencia	Media turno	sd Turno	min Turno	max Turno
Mapa 1	32.19%	57.73	60.37	3	215
Mapa 2	80.00%	18.49	7.92	9	77
Mapa 3	73.75%	22.04	18.43	5	113

Table 4: Resultados A* 1.

- Genético:

Mapa	Tasa de supervivencia	Media turno	sd Turno	min Turno	max Turno
Mapa 1	31.25%	74.17	52.00	5	198
Mapa 2	53.75%	36.65	28.04	11	111
Mapa 3	53.44%	32.59	24.75	6	96

Table 5: Resultados Genético 1.

Resultados experimento 2 (80 agentes)

- BFS:

Mapa	Tasa de supervivencia	Media turno	sd Turno	min Turno	max Turno
Mapa 1	13.02%	186.06	34.79	125	245
Mapa 2	15.84&	89.47	19.35	37	119
Mapa 3	22.17%	86.59	20.36	43	122

Table 6: Resultados BFS 2.

- UCS:

Mapa	Tasa de supervivencia	Media turno	sd Turno	min Turno	max Turno
Mapa 1	13.02%	179.94	31.23	119	242
Mapa 2	19.97%	87.24	23.78	39	122
Mapa 3	24.84%	78.70	23.21	39	122

Table 7: Resultados UCS 2.

- Búsqueda Greedy:

Mapa	Tasa de supervivencia	Media turno	sd Turno	min Turno	max Turno
Mapa 1	13.16%	109.45	49.87	39	179
Mapa 2	14.97%	66.21	27.02	28	107
Mapa 3	17.61%	57.39	24.32	27	107

Table 8: Resultados Búsqueda Greedy 2.

- A*:

Mapa	Tasa de supervivencia	Media turno	sd Turno	min Turno	max Turno
Mapa 1	9.33%	119.33	50.67	27	179
Mapa 2	23.12%	57.09	26.02	27	107
Mapa 3	29.78%	50.44	23.78	25	104

Table 9: Resultados A* 2.

Discusión resultados

Para el experimento 1 podemos notar mayores tasas de supervivencia, dado que al tener menos agentes hay menor congestión, implicando menos

turnos para la salida y mayores probabilidades de llegar a la meta sin que el fuego los alcance. A* fue en general el mejor respecto a tasa de supervivencia y turnos, tuvo resultados similares a Greedy pero esto nuevamente se justifica por la poca congestión. Respecto al experimento 2, la tasa baja de supervivencia baja considerablemente y las medias de los turnos suben, manteniendose A* como la mejor algoritmo respecto a tasa de supervivencia y turnos. Parece curioso que el genético no tenga resultados tan similiares con el A*, dado que con 4 agentes en un mapa con varias rutas, el orden en que se ejecuta A* no es relevante. Probablemente con más iteraciones se parezca a A*, descartando variaciones estocásticas.